Documento de Especificação de Requisitos
Sistema de Gestão de Estoque

Esse documento foi desenvolvido por Lucas Emmanuel em 09/04/2026 e tem como objetivo levantar requisitos do trabalho 01 de POO, além disso, serve como um MVP para o desenvolvimento do "Sistema de Gestão de Estoque".

1. Informações Gerais do Projeto

- **Objetivo:** Desenvolver um software Orientado a Objetos (Java) para a gestão interna de uma loja, com foco principal no controle de estoque.
- **Formação:** O trabalho deve ser feito obrigatoriamente em grupos de 2 integrantes.
- **Avaliação Total:** 25 pontos, divididos em duas etapas.
- **Formato de Envio:** Via Moodle. O arquivo compactado deve seguir estritamente o padrão de nomenclatura: POO T1 Nome1 e Nome2 (para a Parte 1) ou POO T2 Nome1 e Nome2 (para a Parte 2). O envio **fora do template acarreta penalidade** de 20% da nota.
- **Apresentação:** O código fonte será apresentado apenas para o professor em momento a ser acordado.
- **Penalidades Críticas:** Cópias recebem nota zero. Atraso de 1 dia corta a nota pela metade. Mais de 1 dia de atraso não será aceito.

1.1 Cronograma de Entregas

Data de Entrega	Descrição Entrega
10/05/2026	Entrega 1: Envio do código pelo Moodle e apresentação.
02/07/2026	Entrega 2: Envio do projeto no Moodle contendo o código, backup do banco de dados, tutorial de instalação e apresentação.

2. Restrições Técnicas Rigorosas

- **Aderência ao Conteúdo:** É estritamente **proibido utilizar códigos, bibliotecas ou técnicas não ensinadas em sala de aula.** O descumprimento gera decréscimo automático de nota, independentemente de você saber explicar o código ou não.
- **Interface Gráfica:** N**ão é permitido o uso de interfaces gráficas visuais** como javax.swing, JavaFX ou similares em nenhuma etapa do trabalho.

2.1 Especificações da Parte 1 (15 pontos)

- **Entrada e Saída:** Deve ser feita **exclusivamente via console** (utilizando Scanner) e caixas de diálogo (JOptionPane).
- **Persistência (DAO em Memória)**: O uso de banco de dados é proibido nesta etapa. Os dados devem ser **armazenados na memória**, utilizando o padrão DAO (Data Access Object) adaptado para memória.
- **Estrutura de Dados**: É totalmente **proibido utilizar o Collections Framework do Java (Listas, Mapas, Conjuntos, como ArrayList, HashMap, etc.)**. (Atenção: Isso significa que você **precisará usar Arrays primitivos** [] e controlar o tamanho manualmente na Parte 1).
- **Relatórios:** Devem ser exibidos em tela utilizando **System.out.println ou JOptionPane**.

2.2 Especificações da Parte 2 (10 pontos)

- **Persistência:** O projeto passará a **utilizar Banco de Dados**. A conexão deve ser feita exatamente como ensinado pelo professor.
- **Estrutura de Dados:** O **uso do Collections Framework (Listas, Mapas, etc.)** passa a ser obrigatório.
- **Relatórios:** A **saída dos relatórios** deverá ser gerada em **formato .pdf.**

3. Arquitetura do Projeto (Padrão MVC + DAO)

Para garantir que o projeto desenvolvido no IntelliJ abra perfeitamente no NetBeans do professor, o projeto não deve depender de gerenciadores de dependência complexos se não foram ensinados (como Maven/Gradle, a não ser que tenha sido visto em aula). A estrutura de pacotes recomendada é:

- **model:** Classes de entidade (Pessoa, Produto, Pedido, etc.).
    - O que ela faz: Guarda o estado (os dados) dos objetos durante a execução. **Ela contém apenas os atributos (sempre privados), construtores e os métodos de acesso (Getters e Setters), ou seja, Java Bean**
    - O que ela NÃO faz: **Não tem lógica de roteamento, não imprime absolutamente nada na tela** (zero Scanner ou System.out.print), não conhece as outras camadas e não faz a menor ideia de como ser salva na memória ou no banco (isso é problema exclusivo do dao).
- **dao:** Onde vão ficar os arrays fixos que vão simular o seu banco de dados (Ex: ProdutoDAO, ProdutoDAOMemoria).
    - É o único lugar que conhece os Arrays e manipula a memória.
    - O que ela faz: **Recebe o objeto do Controller,** encontra um espaço vazio no Array estático, insere o objeto e avisa o Controller que deu tudo certo.
- **controller:** Lógica de negócio, cálculos de estoque e controle do calendário.
    - É o meio de campo. **Não tem interface gráfica, nem acesso direto ao "banco"**.
    - O que ela faz: **Recebe os dados da View,** cria a instância do objeto (ex: Product novoProduto = new Product(nome, preco);) e manda o DAO salvar:

produtoDao.salvar(novoProduto);. Se der certo, **retorna uma mensagem de "Sucesso" para a View exibir**.

- **view:** Classes responsáveis pelos menus de JOptionPane e Scanner.
  - É o **único lugar onde deve existir JOptionPane ou Scanner**.
  - O que ela faz: **Mostra o menu principal.** O usuário escolhe "1 - Cadastrar Produto". A view pede o nome e o preço. Em seguida, ela chama o Controller: produtoController.cadastrarProduto(nome, preco);

4. Entidades e Modelagem de Dados (Diagrama de Classes UML, pág 6)

Todos os cadastros requerem operações de CRUD (Create, Read, Update, Delete). Todas as entidades devem possuir uma **data_criacao (inserida apenas na criação)** e uma **data_modificacao (atualizada a cada alteração feita)**.

- **Pessoa:** id, nome, nascimento, documento, data_criacao, data_modificacao.
- **Usuário:** id, pessoa (relacionamento), login, senha, data_criacao, data_modificacao.
- **Produto:** id, nome, descricao, preco_venda, ativo, data_criacao, data_modificacao.
- **Carrinho**: id, id_usuario, status (ABERTO, FECHADO, CANCELADO, EXPIRADO), data_criacao, data_modificacao. Um usuário pode ter um histórico com vários carrinhos.
- **Itens_Carrinho**: id, id_carrinho, id_produto, quantidade, preco_unitario (o preço deve ser congelado no momento da adição), data_criacao, data_modificacao.
- **Pedido**: id, id_usuario, id_cupom (opcional), status (CRIADO, PAGO, SEPARACAO, ENVIADO, ENTREGUE, CANCELADO), valor_total, forma_pagamento, data_criacao, data_modificacao.
- **Itens_Pedido**: id, id_pedido, id_produto, quantidade, preco_unitario, subtotal, data_criacao, data_modificacao.
- **Movimentação_Estoque**: id, id_produto, quantidade, tipo (ENTRADA, SAIDA, AJUSTE), valor_unitario, data_criacao/entrada, data_modificacao.
- **Cupom**: id, codigo, tipo_desconto (FIXO, PERCENTUAL), valor_desconto, valor_minimo_pedido, data_validade, ativo, data_criacao, data_modificacao.
- **Entrega**: id, id_pedido, status (preparando, enviado, em_transito, entregue, cancelada), transportadora, codigo_rastreio, data_envio, data_entrega, data_criacao, data_modificacao.

5. Regras de Negócio

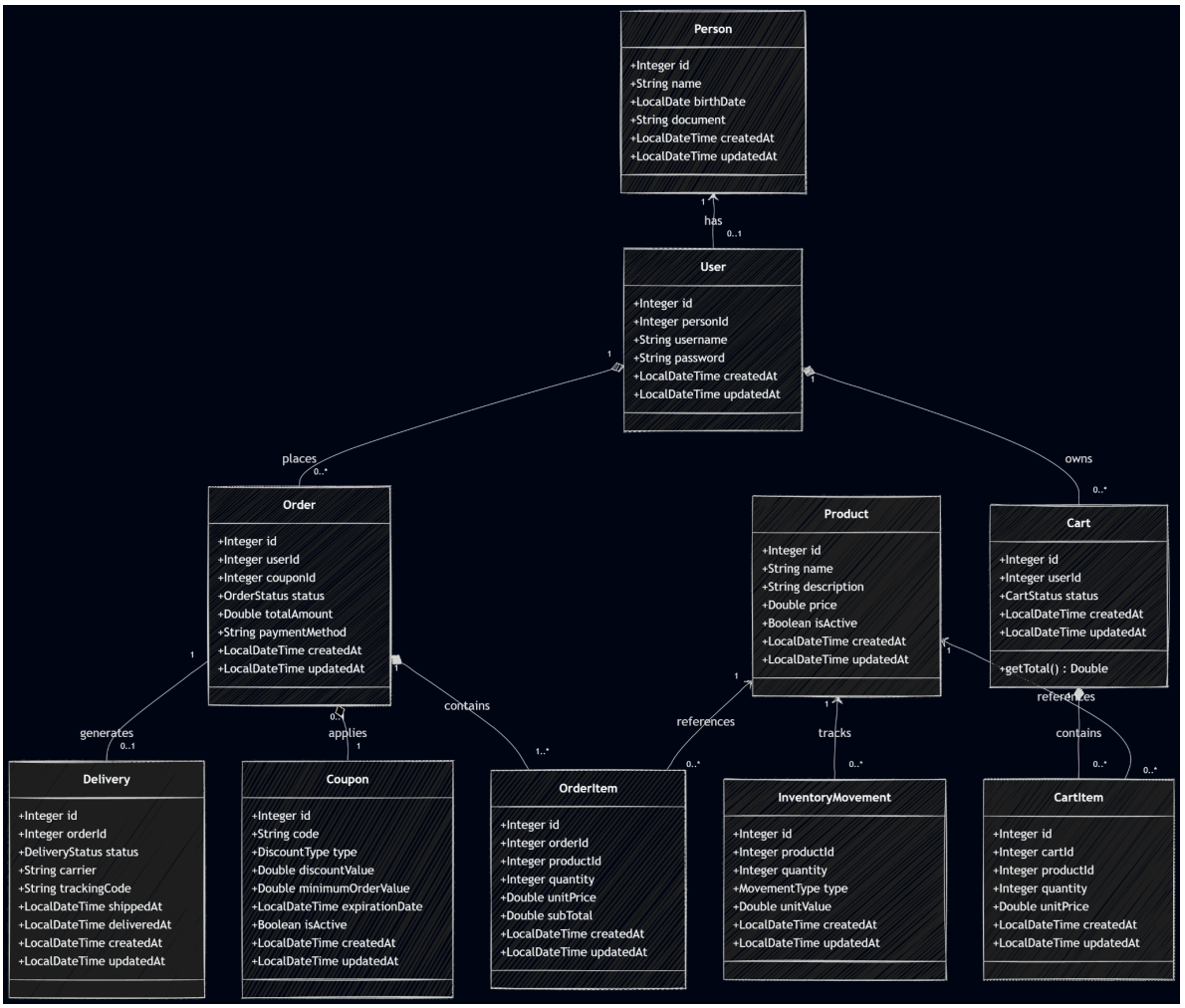
1. **Fluxo Inicial:** O programa deve iniciar exibindo a lista de produtos disponíveis para venda e oferecendo a opção de login.
2. **Fluxo de Compra:** O usuário pode adicionar produtos a um carrinho ou comprar diretamente. Adicionar ao carrinho gera os registros CARRINHO e ITENS_CARRINHO, que podem ser finalizados posteriormente. Finalizar a compra (direta ou pelo carrinho) gera os registros de PEDIDO, ITENS_PEDIDO e MOVIMENTACAO_ESTOQUE.
3. **Controle de Estoque:** O estoque de um produto pode ser calculado com base nas movimentações ou ter seu valor armazenado diretamente (o grupo pode escolher a abordagem).

4. **Descontos:** No fechamento do pedido, deve haver a opção de aplicar um cupom. É necessário cadastrar cupons de teste e validar pedidos com e sem desconto.
5. **Simulador de Tempo (Calendário):** O sistema deve possuir um calendário interno. O Administrador precisa ter acesso a um menu para alterar arbitrariamente a "data atual" do sistema.
    a. **Regra de Carrinho:** Carrinhos expiram em 24 horas. Se a data do calendário avançar 24h, os carrinhos inativos devem ser limpos/expirados.
    b. **Regra de Entrega:** 24 horas após a criação do pedido, o status deve mudar para ENVIADO. 48 horas após o pedido, o status deve mudar para ENTREGUE.

6. Níveis de Acesso e Relatórios

- **Administrador:**
    - Acesso total aos cadastros e ao menu de manipulação do tempo.
    - Relatório de vendas (filtrável por período, cliente ou status).
    - Relatório de faturamento (diário, mensal, anual).
    - Relatório de pedidos (abertos, pagos, cancelados).
- **Usuário Comum:**
    - Acesso apenas ao seu próprio Relatório de pedidos.

Diagrama de Classes (UML)



Referências:

- **PDF BASE:** “2026-1-ADS-Descrição Trabalho 14-03.pdf” disponibilizado no AVA pelo Professor Eduardo Silvestre
- **Geração de UML com o Mermaid Live Editor:** https://mermaid.live/